

Análise Comparativa de Desempenho e Escalabilidade: Abordagens Sequenciais e Paralelas na Extração de Dados via PokéAPI

Raphaela Samille Ramalho de Oliveira¹

¹Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE)
Campus Belo Jardim – PE – Brasil

rsro@discente.ifpe.edu.br

Abstract. *This report investigates the performance of different concurrency models in Python for retrieving and storing images from the PokéAPI. The study compares a sequential baseline against threading, multiprocessing, and concurrent.futures architectures. By analyzing execution times under various workloads (100, 500, and 1,000 images) and worker counts (2, 4, and 8), we quantify the impact of parallelism on I/O-bound systems. Results indicate that thread-based pooling provides the most consistent scalability, significantly reducing latency.*

Resumo. *Este relatório investiga o desempenho de diferentes modelos de concorrência em Python para a recuperação e armazenamento de imagens da PokéAPI. O estudo compara uma linha de base sequencial contra arquiteturas de threading, multiprocessing e concurrent.futures. Ao analisar os tempos de execução sob várias cargas de trabalho (100, 500 e 1.000 imagens) e contagens de workers (2, 4 e 8), quantificamos o impacto do paralelismo em sistemas I/O-bound. Os resultados indicam que o pool baseado em threads oferece a escalabilidade mais consistente, reduzindo significativamente a latência.*

1. Contexto e Objetivos

A coleta automatizada de dados via APIs RESTful é uma tarefa comum na Engenharia de Software, mas que enfrenta o gargalo da latência de rede. A PokéAPI, que serve dados massivos sobre Pokémon, foi utilizada neste projeto como cenário de teste para operações do tipo **I/O-bound**. O objetivo é baixar as imagens (*sprites*) frontais de subconjuntos de Pokémons e armazená-las em disco.

O foco técnico deste trabalho é comparar uma implementação sequencial contra três abordagens paralelas disponíveis na linguagem Python: *threading*, *multiprocessing* e *concurrent.futures*. A análise visa medir como a distribuição de tarefas entre múltiplas unidades de execução (*workers*) otimiza o tempo total de download, reduzindo o tempo de ociosidade do processador enquanto aguarda respostas do servidor.

2. Metodologia e Implementação

A experimentação foi conduzida em um notebook Samsung Galaxy Book 4, equipado com um processador de 10 núcleos físicos e 12 lógicos. O sistema foi desenvolvido em Python 3.14, utilizando as bibliotecas *requests* para chamadas HTTP e *os* para manipulação de arquivos.

Para cumprir os requisitos do projeto, foram implementados scripts que realizam o download de volumes de 100, 500 e 1.000 imagens. Nas abordagens paralelas, foram testados cenários com 2, 4 e 8 *workers*. Cada configuração foi executada 10 vezes consecutivas, e o tempo médio foi registrado para garantir a integridade estatística dos dados, mitigando variações momentâneas na conexão de rede.

3. Resultados Obtidos

Os tempos registrados no terminal foram organizados na Tabela 1, permitindo a comparação direta entre os métodos e a escalabilidade conforme o aumento de *workers*.

Tabela 1. Tempos Médios de Execução (10 repetições por cenário)

Método	Workers	100 Imagens	500 Imagens	1.000 Imagens
<i>Sequencial</i>	1	63.32s	1272.96s	1431.27s
<i>Threads</i>	2	35.28s	415.97s	580.67s
<i>Multiprocessing</i>	2	43.04s	235.49s	603.31s
<i>Futures</i>	2	30.81s	426.46s	638.34s
<i>Threads</i>	4	26.43s	230.55s	478.88s
<i>Multiprocessing</i>	4	38.51s	252.33s	423.69s
<i>Futures</i>	4	27.77s	235.19s	507.43s
<i>Threads</i>	8	22.22s	157.81s	981.64s
<i>Multiprocessing</i>	8	45.23s	146.10s	3596.37s
<i>Futures</i>	8	24.69s	205.84s	405.40s

4. Discussão Técnica

A análise dos resultados demonstra que o paralelismo reduziu drasticamente o tempo de execução em comparação ao método sequencial. No entanto, o comportamento dos métodos variou conforme a carga.

Para volumes de 100 e 500 imagens, o aumento de *workers* resultou em ganhos de performance consistentes, com o *Multiprocessing* e *Threads* atingindo seus melhores tempos com 8 *workers*. Contudo, no cenário de 1.000 imagens com 8 *workers*, observou-se uma anomalia severa no *Multiprocessing* (3596.37s) e uma perda de eficiência nas *Threads* (981.64s).

Esse fenômeno sugere que, em cargas muito altas, o custo de coordenação de muitos processos no Windows, somado ao possível *rate limiting* da PokéAPI ao receber conexões simultâneas agressivas, acaba prejudicando o desempenho. O método *Concurrent Futures* provou ser o mais estável para a carga máxima de 1.000 imagens com 8 *workers*, mantendo um tempo de 405.40s.

5. Conclusão

O projeto confirmou que, para tarefas baseadas em rede (I/O-bound), a programação paralela é indispensável. O hardware utilizado (Galaxy Book 4) respondeu bem ao escalonamento até 4 *workers*, mas apresentou gargalos com 8 em cargas extremas. Conclui-se que o *Concurrent Futures* oferece o melhor equilíbrio entre simplicidade de implementação e estabilidade de performance para sistemas de coleta de dados em larga escala.